Hundi — Complete Build Handoff

HashKey Chain Horizon Hackathon, Mumbai Edition · Track 2: Agent Payments & A2A Settlement

For the lead builder. Read the whole document once. Then keep Sections 5, 6, 7 and 12 open while you build.

---

0. Read this first

What this is. A complete brief for building **Hundi**: a system where a buyer's AI agent accepts an invoice, lenders advance cash to the supplier within minutes, and the buyer's later payment through **HSP** (HashKey Settlement Protocol) lands in a per-invoice vault that repays the lender first.

Three rules for the whole project.

1. **Verify before you trust.** Facts marked **[VERIFY]** come from secondary sources (competitor READMEs, news articles, my own reasoning). Check them against official docs before building on them. The official HSP docs are at https://hashkey-merchant.github.io/en/index.html (JavaScript-rendered, open in a browser) and the HashFans user manual at https://hashfans.io/.
2. **Build in layers.** L0 and L1 (Section 13) make a submittable project. Do not start L2 or L3 until L0 and L1 work end to end on mainnet.
3. **Honesty is a feature.** Judges read many submissions. Ours must say exactly what is real, what is simulated, and what is roadmap (Section 15).

Hard dates (IST). Submission deadline **Sun 1 Nov 2026, 21:30** (Mon 2 Nov 00:00 Beijing). Demo Day **Wed 4 Nov**. Winners **Thu 5 Nov**. Suggested internal targets: feature freeze 1 Nov 15:00, submit by 18:30, leaving three hours of buffer.

---

1. Hackathon facts and hard rules

Item	Fact	Status
Event	HSK Chain Horizon Hackathon, Mumbai Edition, 23 Oct – 5 Nov 2026	Official page

Item	Fact	Status
Our track	Track 2: Agent Payments & A2A Settlement. Prize 5,000 USDT: 2,500 / 1,500 / 1,000	Official page
HSP	**Mandatory** for Track 2	Official page
Track 2 topic	A2A scenario with HSP's API: the Payment Agent builds a Payment Mandate from the payment conditions sent by the receiving agent, signs with its wallet, then executes the on-chain payment through HSP's existing Gateway	Official page
Originality	Must be built or rebuilt during the hackathon period	Official page
Mainnet	The official page says projects that have not completed **HSK Chain mainnet** deployment forfeit eligibility for demo selection. The event text you were given says testnet is fine until you win. **These conflict.** Ask the organisers in the HSK Chain Telegram and plan for mainnet regardless	**[VERIFY]**
Extra points	Using officially recommended products and frameworks earns extra points. HSP is the obvious one. Chainlink CCIP is also in the official resource list	Official page
Judges	Five: three from ecosystem partners, two from HSK Chain. Three attend in person. Criteria not published; the page asks for strong completeness, technical maturity and innovation	Official page
Chains	Testnet chain ID **133**. Mainnet chain ID **177**. Mainnet RPC `https://mainnet.hsk.xyz`	Secondary sources **[VERIFY]**
Tools	Faucet https://hskchain.net/faucet · Docs https://hskchain.net/docs · Explorer https://hsk.blockscout.com/	From your event text
Submission	DoraHacks BUIDL: https://dorahacks.io/hackathon/hskchainindia/detail	Official page

What past winners tell us. In the Hong Kong edition, the PayFi first place went to FlowLink (on-chain invoicing, payment links, AI payment agents) and second to hashpay (payroll with receipt-backed advances and attestations). In Japan, Omamorai (screens every payment before it executes, opening with a ¥324 billion fraud statistic) was a finalist-profile project, and Helm, Maneki AI, MiuroAI and AllScale Agentic Commerce Gateway placed. The pattern: **a real financial workflow, a quantified problem, guardrails or human approval, a working on-chain demo, and credit or invoicing built on HSP receipts has already placed.**

---

2. The problem statement

> **Indian MSMEs are owed about ₹8.1 lakh crore in delayed payments. They wait around 73 days for money the law says is due in 45, and the platforms built to fix this do not reach most of them. Hundi turns a buyer-accepted invoice into instant cash and repays the lender automatically through HSP, with the repayment path fixed before the loan is made.**

Claim	Source and confidence
About ₹8.1 lakh crore stuck in delayed MSME payments	Economic Survey 2025-26 as reported by several outlets. **[VERIFY against the Survey itself before pitching]**
Average 73 days to clear invoices vs 45-day legal limit	A vendor report (Recordent) cited by a fintech news site. **Label it vendor-sourced**
Section 43B(h): since April 2024 a buyer loses the tax deduction if an MSME is unpaid beyond the 45-day limit	Secondary articles **[VERIFY]**
TReDS invoice discounting grew from about ₹40,000 crore (FY22) to ₹3.47 lakh crore (FY26)	Secondary article **[VERIFY]**. Use as proof the model works
TReDS is open only to RBI-regulated entities and onboarded buyers	Secondary article **[VERIFY]**. Say Hundi *complements* TReDS for the long tail
Typical invoice-discounting rate 5–8% a year	One secondary article. **[VERIFY]**; never present as a guarantee

Why agents, HSP and a chain are all needed.

1. **The buyer's acceptance is what makes an invoice financeable**, and AI agents are increasingly the ones reading invoices. An agent that reads untrusted text is an attack surface: researchers have shown hidden web prompts tricking agents into sending crypto payments. So our acceptance gate is deterministic code; an LLM can flag risk but can never approve.
2. **HSP payments are final.** HSP settles one relayed transfer to the cart's `pay_to`, with no escrow, refunds, splits or disputes (as described by a competitor's README, **[VERIFY]**). So we bind `pay_to` to a per-invoice vault whose code repays the lender first.
3. **Cross-border invoices settle naturally in stablecoins.** The wedge is Indian exporters invoicing overseas buyers. This answers the question every judge will ask: why a chain, when the statistic is in rupees?

What we do not claim. We do not claim to solve ₹8.1 lakh crore. We do not claim legal effect, regulatory approval or a TReDS replacement. We demonstrate a mechanism, with real transactions, and say what is simulated.

---

3. The solution in simple words

A small Indian exporter ships goods to an overseas buyer and issues a $10,000 invoice due in 73 days.

1. The exporter's **seller agent** sends the invoice and payment conditions.
2. The buyer's **buyer agent** checks it with strict rules (does it match the purchase order and delivery? is the vendor approved? is this a duplicate?) and, if everything passes, **signs an acceptance**.
3. The **liquidity pool** (lenders' money) advances, say, 90% of the invoice to the exporter within minutes, minus a small bond.
4. At the due date, the buyer agent pays through **HSP**. The payment goes to a **vault address that was fixed when the invoice was accepted**. The vault's code pays the lender first, then the protocol fee, then the rest to the exporter.

The core insight. HSP settles a signed payment to whatever address the merchant's cart names. We make that address a vault computed from the loan terms (a CREATE2 address that need not exist yet). So **a lender can verify, before lending anything, that the buyer's**

future payment can only arrive at an address that repays them first. The invoice becomes a verifiable financial instrument, not just a promise in a database.

Glossary. *Receivable*: money a buyer owes a seller. *Advance*: cash given now against it. *Discount fee*: the lender's charge for the wait. *Maturity / due date*: when the buyer pays. *Waterfall*: the fixed order in which incoming money is split. *Acceptance*: the buyer's signed confirmation that the invoice is valid. *Mandate*: AP2's signed permission for a payment (Intent, Cart, Payment). *Vault*: our per-invoice contract.

---

4. Competition and positioning

Public Track 2 repos seen on GitHub (their final submissions may differ):

Project	What it does
AgentTab	Pay-per-request vouchers, batch settlement, HSP top-ups; mainnet
Kohaku	Clearing house; every net debt is an HSP order; A2A + MCP; roadmap mentions credit lines
Payminto	Netted HSP settlement, mandate-bound vouchers, MCP wallet
Leash	Budget-bound wallet and escrowed agent hiring
Proviso	Conditional settlement: HSP payment lands in a CREATE2 escrow derived from deal terms; AP2 over A2A; ERC-8004; 73 contract tests, 42 SDK tests, adversarial audit, fork tests, eval set, JUDGES.md / CLAIMS.md / PROOF.md
Likeness, VeriQuant, ShunyaGrid	Licensing payments, quant terminal, compute settlement

Positioning. Everyone else builds payment pipes. **Hundi builds credit on top of the pipe.** Kohaku lists credit lines as roadmap; we ship it. hashpay (Hong Kong, 2nd place PayFi) validated receipt-backed advances in payroll; we take the idea to B2B trade invoices, a far larger market.

Do not lead with AP2, A2A, MCP or ERC-8004. Others already use them. They are table stakes; the differentiator is the financing primitive and the rigor of execution. **Proviso is the quality bar**: match its tests, docs and honesty, then beat it on category and problem scale.

---

5. System architecture

5.1 Components

Component	Technology	Job
`ReceivableFactory`	Solidity, Foundry	Deploys per-invoice vaults at addresses computed from the terms (CREATE2 clones)
`ReceivableVault`	Solidity	Holds the waterfall: Open → Funded → Settled or Defaulted
`AcceptanceRegistry`	Solidity, EIP-712, ERC-1271	Records buyer-signed acceptances; blocks duplicate invoices
`LiquidityPool`	Solidity, ERC-4626	Lenders deposit USDC.e; funds advances within on-chain limits
`ReceivableNote` (L2)	ERC-721	Transferable lender claim; vault pays the current owner
Desk	TypeScript, Fastify, SQLite	HSP merchant of record, webhook handler, keeper, REST API, live event stream
Agents	TypeScript, A2A, AP2 types	Seller, buyer, underwriter; deterministic policy code with an LLM for extraction and memos only
MCP server	TypeScript	Lets any agent issue, accept, finance and pay invoices
Web	Next.js	Live dashboard: timeline, waterfall, attack panel, evidence pack

5.2 Lifecycle, step by step

1. **Issue.** The seller agent creates a canonical invoice JSON (seller, buyer, token, amount, due date, goods/service hash, PO reference, invoice number) and its `invoiceHash`.
2. **Terms.** The desk builds `Terms` (Section 6.1), computes `termsHash`, and computes `vault = factory.predictVault(terms)`.
3. **Accept.** The buyer agent runs `verifyInvoice` (Section 8). On pass, it signs an EIP-712 `Acceptance` that includes `vault`, `termsHash`, amount and due date. Anyone can submit it to `AcceptanceRegistry.accept`.
4. **Create vault.** `factory.createVault(terms)` deploys the vault at the predicted address (requires a recorded acceptance).
5. **Underwrite.** The underwriter agent proposes advance rate and fee **within the pool's on-chain bounds** and writes a risk memo. Deterministic numbers; LLM text only explains them.
6. **Advance.** `pool.advance(...)` checks the registry, bounds and exposure caps, then calls `vault.fund`. The seller receives advance minus bond.
7. **Mature.** At or near the due date, the desk creates the **HSP order just-in-time** (HSP cart expiry is unknown, **[VERIFY]**) with `pay_to = vault` and amount = face value.
8. **Pay.** The buyer agent runs `verifyCart` (code, not a prompt): merchant signature, cart hash, `pay_to == predictVault(terms from the stored acceptance)`, amount, token, due date. If it passes, it signs the payment authorization and submits the AP2 Payment Mandate to HSP.
9. **Settle.** HSP's relayer moves USDC.e to the vault. The desk's webhook handler confirms it on-chain; anyone calls `vault.settle()`. The waterfall pays out.
10. **Record.** Events update the dashboard and the buyer's on-chain payment record. If unpaid past the default window, `declareDefault()` runs.

5.3 Vault state machine

`Open` (deployed, unfunded) → `Funded` → `Settled` (on time or late) or `Defaulted`. `Open` → `Cancelled` if the acceptance expires unfunded. If the buyer pays while still `Open` (prepaid), `settle()` pays the seller in full and the vault can never be funded afterwards.

5.4 Money math (worked example, USD)

Face value $10,000; advance 90%; fee 8% a year; tenor 73 days (exactly 0.2 of a year); bond 2% of advance; protocol fee 0.1% of face.

Line	Calculation	Amount
Advance	90% × 10,000	$9,000
Bond held	2% × 9,000	$180
Seller receives at funding	9,000 − 180	**$8,820**
Discount fee	9,000 × 8% × 73/365	$144
Protocol fee	0.1% × 10,000	$10
Lender due at maturity	9,000 + 144	$9,144
Seller residual at maturity	10,000 − 9,144 − 10	$846
Bond returned		$180
Seller total	8,820 + 846 + 180	**$9,846**
Total cost to seller		**$154 (1.54%)**

Check: 9,846 + 144 + 10 = 10,000. Make the dashboard show this table live, with days-to-cash falling from 73 to minutes.

5.5 Late, partial and default rules

- **Late:** after the due date, late interest accrues to the lender on the advance at `lateAnnualBps`, paid ahead of the protocol fee.
- **Partial payment:** all funds go to the lender until lender due is covered; the vault stays `Funded` until fully paid or defaulted.
- **Overpayment:** any amount above face value goes to the seller.
- **Default:** after `dueDate + defaultSeconds`, anyone may call `declareDefault()`. The bond is forfeited to the lender up to the shortfall. The buyer's negative record is written. The evidence pack is generated.
- **Pull fallback:** payments are pushed to recipients, but if a transfer fails (for example a blocklisted address), the amount waits in the vault for `claim()`. One bad recipient must never freeze the others.

5.6 Design rules

- **Settle in USDC.e only.** With the EIP-3009 authorization the recipient is inside the payer's signature. For USDT's Permit2 path the recipient is not, which weakens the guarantee **[VERIFY]**.
- **Label every settlement's rail honestly:** *HSP gateway* only when the funding transfer was sent by HSP's relayer (a competitor lists `0x1aaA5E4371aAdf96bEfb5bd2Af4d896FD16915Ea`, **[VERIFY]**), otherwise *self-relayed*.
- **Reputation only from funded receivables.** A competitor's audit found zero-capital vaults could poison reputation. We write records only after real funding and settlement, bound to the actual parties.
- **No upgradeable contracts** for the vault. Immutable is easier to audit and to claim.

---

6. Smart contract specification

6.1 The `Terms` struct (immutable, hashed into the vault address)

`seller, buyer, token (USDC.e), faceAmount, dueDate, invoiceHash, advanceBps, feeAnnualBps, lateAnnualBps, bondBps, protocolFeeBps, graceSeconds, defaultSeconds, pool, protocolTreasury`

`termsHash = keccak256(abi.encode(terms))`. The vault is a deterministic clone: `salt = termsHash`, address from `Clones.predictDeterministicAddress`. Because the address commits to every term, **no one can later change the waterfall without changing the address the buyer signed.**

6.2 Contracts and functions

AcceptanceRegistry

- `accept(Acceptance a, bytes sig)`: verifies EIP-712 signature (ECDSA, or ERC-1271 for contract wallets via OpenZeppelin `SignatureChecker`), expiry, unused nonce, unused `invoiceKey = keccak256(seller, buyer, invoiceNumberHash)`.
- `isAccepted(termsHash)`, `acceptanceOf(termsHash)`.
- `Acceptance`: `buyer, seller, token, faceAmount, dueDate, invoiceHash, termsHash, vault, nonce, expiry`.

ReceivableFactory

- `predictVault(Terms)` (pure view), `createVault(Terms)` (requires acceptance; permissionless; atomic initialize).

ReceivableVault

- `fund(address lender, uint256 advance)`: pool only, once. Pays seller `advance − bond`.
- `settle()`: permissionless; reads token balance; applies waterfall; idempotent and monotonic.
- `declareDefault()`: permissionless after `dueDate + defaultSeconds` if unsettled.
- `claim(address)`: pull fallback. `cancel()`: if `Open` and acceptance expired.
- `amountDue()` view: lender due including late interest at the current time.

LiquidityPool (ERC-4626)

- Standard `deposit / withdraw / redeem`. Withdrawals limited to idle liquidity.
- `advance(Terms, Acceptance, sig)`: checks registry, `advanceBps <= maxAdvanceBps`, `feeAnnualBps >= minFeeBps`, tenor `<= maxTenorDays`, per-buyer and per-seller exposure caps, not paused.
- `guardian` can pause new advances only. Never funds withdrawal.

ReceivableNote (L2)

- ERC-721 minted at funding to the lender. The vault pays `ownerOf(tokenId)`. Transfer is the secondary market.

6.3 Events

`InvoiceAccepted`, `VaultCreated`, `Advanced`, `Settled(lenderPaid, protocolPaid, sellerPaid, daysLate)`, `DefaultDeclared`, `BondForfeited`, `Claimed`. Every dashboard element must be reconstructable from events alone.

6.4 Invariants (write these as Foundry invariant tests before the code)

1. **Conservation:** tokens that enter a vault equal tokens that leave plus tokens waiting in `claim`.
2. **Single financing:** a `termsHash` is funded at most once; an `invoiceKey` is accepted at most once.
3. **Lender first:** lender due is always paid before protocol fee and seller residual.

4. **Bounded advance:** advance never exceeds `advanceBps × face` or pool exposure caps.
5. **Valid acceptance only:** state moves to Accepted only with a valid buyer signature.
6. **Address determinism:** the vault address is a pure function of the terms.
7. **Monotonic state:** no state moves backwards; `settle()` cannot double-pay.
8. **No frozen funds:** a failing recipient never blocks others.
9. **Reputation integrity:** records are written only for funded receivables.

6.5 Security notes

- Use `nonReentrant`, checks-effects-interactions, and `SafeERC20`.
- Tokens sent to the vault **before** it is deployed are fine: the vault reads `balanceOf` at settle time.
- Treat dust and unsolicited transfers as overpayment to the seller.
- Use OpenZeppelin for clones, ERC-4626, ERC-721, ECDSA and SignatureChecker. Do not hand-roll cryptography.
- Pin Solidity and OpenZeppelin versions; commit `foundry.lock` or equivalent.

---

7. HSP integration specification

What we know (from a competitor's README and the official topic text; [VERIFY] everything against official docs).

- HSP is AP2-aligned. The merchant creates a **Cart Mandate** (an order) and signs it; the payer's wallet signs a **Payment Mandate**; HSP's gateway verifies and executes the payment on-chain.
- Merchant API call to create orders: `POST /api/v1/merchant/orders`; payer submission: `POST /api/v1/payment/pay-mandate`.
- Requests are HMAC-SHA256 signed; the cart carries an ES256K `merchant_authorization` signature; webhooks are signed and must be verified.
- For USDC.e the payer signs an **EIP-3009** `transferWithAuthorization`; HSP's relayer submits it.
- The transfer is **final**: no escrow, refunds, splits or disputes at the HSP layer. All of that logic must live in our vault.

- `pay_to` can be a counterfactual contract address. A competitor's design depends on this. **Test it with a real small payment on day one.**

Adapter modules (`packages/sdk/hsp`).

Module	Responsibility
`merchantClient`	Create orders, sign cart mandates, HMAC requests, fetch order status
`payerClient`	Build AP2 Payment Mandate, sign EIP-3009, submit to `pay-mandate`
`webhook`	Verify signature; idempotent processing keyed by flow or order ID
`reconcile`	Confirm on-chain: token, `Transfer(buyer → vault, amount)`, exact amount, confirmations
`rails`	Label settlement *gateway* or *self-relayed* from the funding tx sender

Modes. `HSP_MODE=gateway` uses real credentials. `HSP_MODE=self-relay` broadcasts the buyer's identical signed authorization ourselves, for testnet, forks and any time credentials are missing. Both modes are valid; they must be **labelled** in the UI and in PROOF.md.

If you do not get HSP credentials in time. Other teams reported the official SDK or coordinator unreachable and built to the wire spec behind one interface. Do the same, write an honest "HSP integration" section, keep asking the organisers, and switch to gateway mode the moment access arrives.

Credentials checklist. Merchant app key and secret, ES256K merchant signing key and public key registered with HSP, webhook URL (public HTTPS), supported token list, order expiry rules, base URL. Keep all in `.env`, never in git.

---

8. Agents, AP2/A2A and MCP

Agent	Role	Notes
Seller agent	A2A merchant role. Issues invoices, requests advances	Publishes an Agent Card at `/.well-known/agent-card.json`
Buyer agent	A2A payment role. Verifies invoices, signs acceptance, pays at maturity	All decisions in code
Underwriter agent	Proposes advance and fee within pool bounds; writes a risk memo	Numbers are deterministic; LLM only explains
Keeper	Triggers `createVault`, `advance`, `settle`, `declareDefault`	All functions are permissionless; anyone could run it
Desk	HSP merchant of record, API, event stream	**Honest limit:** one merchant of record for all sellers

The LLM boundary (the most important rule). The LLM may (a) extract fields from invoice text into a strict JSON schema, (b) write risk memos, and (c) **raise** a risk flag. It may never approve anything. Approval means *every* deterministic check passes, plus a human approval above a configured threshold. All counterparty text is untrusted data: never place it in an instruction position, always validate LLM output against a zod schema, and fail closed.

`verifyInvoice` refusal codes (named, returned to the caller, tested). `SCHEMA_INVALID`, `SIGNATURE_INVALID`, `VENDOR_NOT_ALLOWED`, `OVER_CAP`, `DUPLICATE_INVOICE`, `PO_MISMATCH`, `DELIVERY_NOT_ATTESTED`, `DUE_DATE_OUT_OF_POLICY`, `TOKEN_MISMATCH`, `PAYEE_CHANGED`, `HUMAN_APPROVAL_REQUIRED`, `LLM_RISK_FLAG`.

`verifyCart` refusal codes. `CART_SIGNATURE_INVALID`, `CART_HASH_MISMATCH`, `PAYTO_NOT_VAULT`, `AMOUNT_MISMATCH`, `TOKEN_MISMATCH`, `DUE_DATE_MISMATCH`, `ACCEPTANCE_NOT_FOUND`, `ALREADY_SETTLED`.

Delivery attestation (honest limit). In the prototype, proof of delivery is a signed attestation from a trusted party (simulated logistics agent). State this plainly. Roadmap: GST e-invoice and e-way bill data.

MCP server tools. `issue_invoice`, `accept_invoice`, `request_advance`, `pay_invoice`, `get_receivable`, `list_receivables`. Offer HTTP and stdio transports. This is how other people's agents can use Hundi, which is the legacy move.

---

9. Threat model

Every row needs a test. The right-hand column says which test proves it.

Threat	Defence	Proven by
Prompt-injected or fake invoice	LLM cannot approve; deterministic `verifyInvoice`; fail closed	Eval cases INJ-1 to INJ-3; live demo attack 1
Same invoice financed twice	`invoiceKey` single use; `termsHash` funded once	Invariant 2; live demo attack 2
Payee or bank details swapped (BEC)	Payee fixed inside the signed Acceptance; `PAYEE_CHANGED`, `PAYTO_NOT_VAULT`	Unit tests; live demo attack 3
Cart tampered after merchant signed	`CART_HASH_MISMATCH`, merchant signature check	Unit test
Acceptance replay	EIP-712 domain with chainId and verifying contract, nonce, expiry	Fuzz and unit tests
Griefing by deploying vaults	Address is a pure function of terms; creation requires an acceptance; same result whoever calls	Invariant 6
Reentrancy or hostile token	USDC.e allowlist, `nonReentrant`, checks-effects-interactions	Unit tests with a malicious mock token
Blocklisted recipient freezes payout	Push with `claim()` pull fallback	Fork test with a blocklisted address
Buyer defaults	Bond forfeit, `declareDefault()`, evidence pack, negative record	E2E default scenario
Seller fakes delivery	Delivery attestation plus bond. **Honest limit** in prototype	CLAIMS.md

Threat	Defence	Proven by
Pool drained by bad advances	On-chain bounds, per-buyer and per-seller caps, guardian can only pause new advances	Unit and invariant tests
Reputation poisoning	Records only for funded, settled or defaulted receivables, bound to real parties	Invariant 9
LLM or service outage	Fail closed; nothing is approved by default	Eval negative controls
Buyer agent key compromised	Spend-limited wallet with per-invoice and daily caps (roadmap if not built)	CLAIMS.md
Single merchant of record	**Honest limit**; roadmap is per-seller merchant accounts	CLAIMS.md

---

10. Testing and rigor package

This is where we match and beat the strongest competitor. Numbers are a bar, not a goal: every test must check something real.

Contracts (Foundry).

- Unit tests for every function and every revert.
- **Fuzz tests** on the waterfall: random face value, advance rate, fee, days late. Property: the sum of payouts equals the funds in.
- **Invariant tests** for all nine invariants in Section 6.4, written *before* the implementation.
- **Fork tests** against HSK mainnet using `anvil --fork-url https://mainnet.hsk.xyz` **[VERIFY]**, using the real USDC.e contract.
- Slither static analysis with findings triaged in AUDIT.md.
- Coverage target: at least 90% on vault, factory, registry and pool.
- Aim for 70 or more meaningful tests including fuzz and invariants.

SDK and desk (Vitest). Table-driven tests for `verifyInvoice` and `verifyCart` (every refusal code); HSP adapter against recorded fixtures; webhook signature and idempotency tests; reconcile tests.

The evaluation set (this builds credibility). Create `eval/cases.json` with about 24 invoice cases and an answer key **committed to git before any model runs**:

Category	Cases
Clean, should be accepted	8
Duplicate invoices	3
Inflated amounts	3
Prompt injection in notes or document text	3
Payee or bank change	3
Due-date manipulation	2
PO mismatch	2

Plus 4 negative controls decided by code with no model involved. Run the set twice and keep both result files. Report results honestly, including failures. A published failure with a fix beats a perfect score nobody believes.

Adversarial review. Run a *separate* Claude Code session (fresh context) as the auditor using the checklist in Section 12. Each finding gets a severity, a fix and a regression test in `contracts/test/audit/`. Log everything in AUDIT.md.

End-to-end script. `scripts/e2e.ts` runs: happy path, late payment, default, prepaid, duplicate invoice, injected invoice, payee swap, tampered cart, partial payment. It prints transaction hashes for PROOF.md.

CI. GitHub Actions running `forge test`, `vitest` and lint on every push.

---

11. Repository layout

```
hundi/
  CLAUDE.md
  README.md  JUDGES.md  CLAIMS.md  PROOF.md  AUDIT.md
  docs/
    HANDOFF.md          <- this document, saved as markdown
    DECISIONS.md        <- running log of choices and why
    spec/               <- one file per Section 5-8
  tasks/todo.md         <- current plan, checkable items
```

```
contracts/
  src/  test/  test/audit/  script/  deployments/177.json
packages/
  sdk/                <- HSP adapter, AP2 types, signers, verify functions
  mcp/                <- MCP server (stdio and HTTP)
apps/
  desk/               <- API, agents, keeper, webhook, SQLite
  web/                <- Next.js dashboard
eval/                 <- cases.json, answer key, results
scripts/              <- e2e.ts, deploy helpers
.claude/
  agents/  commands/  settings.json
```

---

12. How to use Claude Code for all of this

Verify against the official docs. Claude Code changes often. The official documentation index is https://code.claude.com/docs/llms.txt and the everyday-workflows page is https://code.claude.com/docs/en/common-workflows.md. Items below marked **[VERIFY]** come from community guides.

12.1 The principle

You are the architect and the reviewer. Claude Code writes; you decide and check. Never merge code you cannot explain, especially signature verification and money movement. Claude can produce confident, plausible, wrong cryptography and invented addresses, so the discipline in 12.9 is not optional.

12.2 Setup

1. Install Claude Code from the official docs and run `claude` inside the repo.
2. Save this handoff as `docs/HANDOFF.md`. Split Sections 5–8 into `docs/spec/*.md` so Claude can read only what a task needs.
3. Create `CLAUDE.md` (12.3), `.claude/agents/` (12.6), `.claude/commands/` (12.7).
4. Set `.claude/settings.json` permissions so Claude can run `forge`, `pnpm` and `git` but cannot read `.env` or run destructive commands. Check the permissions page in the docs.
5. Add `.env` to `.gitignore` *before the first commit*.

12.3 Starter CLAUDE.md

Claude Code reads CLAUDE.md automatically each session. Keep it short (community advice is under about 200 lines **[VERIFY]**) and put long reference in `docs/`. Update it every time Claude makes a mistake.

```
# Hundi - project rules for Claude Code

## What this project is
Hundi finances buyer-accepted invoices on HashKey Chain. HSP pays into a per-invoice vault that repays the lender first. Full brief: docs/HANDOFF.md. Read the relevant section before touching code.

## Layout
contracts/ (Foundry) | packages/sdk (HSP adapter, AP2 types, signers) | apps/desk (API, agents, keeper) | apps/web (dashboard) | packages/mcp | docs/ | eval/

## Commands
- Contracts: cd contracts && forge build && forge test -vv
- SDK: pnpm --filter sdk test
- Desk: pnpm --filter desk test
- Everything: pnpm test:all (must pass before any commit)

## Non-negotiable rules
1. An LLM never approves a payment, acceptance or advance. Approval is deterministic code only.
2. Never invent addresses, token addresses, chain IDs, endpoints or signatures. Take them from config files or official docs and say where they came from. If unknown, stop and ask.
3. Never put secrets in code, tests, logs or commits. Use .env (gitignored). Never print private keys.
4. Money logic and signature code: write the test first, then the code. Use OpenZeppelin. Do not hand-roll crypto.
5. Settlement token is USDC.e only. Contracts are immutable (no proxies).
6. Do not edit files outside your assigned area without asking.
7. Mark anything unverified as [VERIFY] in comments and in CLAIMS.md.

## Workflow
- Non-trivial task: plan first, write the plan to tasks/todo.md, wait for approval.
- Failing test first, then implementation, then run all tests.
- Small commits, one concern each, clear messages.
```

```
- When finished, list what you did NOT verify.

## Style
Solidity 0.8.x pinned, NatSpec on external functions, custom errors.
TypeScript strict, zod at every boundary, no any.
```

12.4 The first session: briefing prompt

Paste this to start. It forces Claude to prove it understood before it writes anything.

```
You are joining a hackathon project called Hundi. Before doing anything:
1. Read docs/HANDOFF.md fully, then CLAUDE.md.
2. Summarise back to me in under 300 words: the problem, the core insight
(pay_to bound to a per-invoice vault), the lifecycle, and the nine
invariants.
3. List every item marked [VERIFY] and propose how to verify each one.
4. Propose a plan for Layer L0 only, as a checklist in tasks/todo.md, listing
the files you will create. Do not write code yet.
Ask me questions if anything is ambiguous. Do not guess protocol details.
```

Correct any misunderstanding *now*. A wrong assumption fixed in the first ten minutes costs nothing; fixed after the contracts exist, it costs hours.

12.5 Parallel sessions

Run several Claude Code sessions at once, each in its own **git worktree** so they never overwrite each other's files (the official workflows page covers worktrees; the `--worktree` flag is from community docs **[VERIFY]**; plain `git worktree add ../hundi-sdk -b feat/sdk` always works). Give each session a file-ownership boundary.

Session	Owns	Starts when
A: contracts	`contracts/`	First. Everything depends on its ABIs
B: sdk and HSP adapter	`packages/sdk/`	After Terms and Acceptance structs are frozen
C: desk and agents	`apps/desk/`	After sdk exposes verify functions
D: web	`apps/web/`	After desk exposes the event stream (mock it earlier)
E: auditor (read-only)	nothing	After each contracts milestone

Session	Owns	Starts when
F: docs and eval	`eval/`, root docs	In parallel from the start

Freeze interfaces early. Write the `Terms` struct, the `Acceptance` struct and the contract ABIs first, generate TypeScript types from them into `packages/sdk`, and let all other sessions code against those types.

12.6 Subagents (custom specialists)

Subagents are markdown files in `.claude/agents/` with a short header (name, description, allowed tools, optional model) and a system prompt **[VERIFY field names in the docs]**. They run in their own context and keep your main conversation clean. Create at least these:

```
---
name: solidity-auditor
description: Adversarial reviewer for Solidity contracts. Use after any
change in contracts/src. Never edits code.
tools: Read, Grep, Glob, Bash
---
You are an adversarial smart-contract auditor. You did not write this code.
Check it against docs/HANDOFF.md Section 6.4 (invariants) and Section 9
(threat model).
For each finding give: severity, location, exploit scenario, suggested fix,
and a failing Foundry test that demonstrates it.
Look especially for: reentrancy, rounding errors in the waterfall, signature
replay, unchecked token transfers, griefing of vault creation, frozen funds.
Report only. Do not modify files.
```

- **spec-checker**: compares code against the spec and lists every deviation.
- **hsp-researcher**: reads official HSP docs and fixtures, answers protocol questions, cites the page.
- **test-writer**: writes tests from the invariants, never touches `src/`.

12.7 Slash commands (reusable prompts)

Markdown files in `.claude/commands/` become commands **[VERIFY]**. Useful ones: `/plan-feature` (plan to tasks/todo.md, no code), `/audit-contracts` (invoke the auditor, write AUDIT.md), `/run-all` (every test suite, summarise failures), `/update-proof`

(collect addresses and transaction hashes into PROOF.md).

12.8 The loop for every task

1. **Plan mode first** for anything non-trivial (cycle modes with Shift+Tab **[VERIFY]**). Ask for the plan, edit it, approve it.
2. **Write the failing test.** Run it and watch it fail.
3. **Implement** until the test passes.
4. **Run everything.** `pnpm test:all`.
5. **Independent review** by the auditor subagent or a fresh session.
6. **Commit** with a clear message. Your git history proves the work was done during the hackathon.
7. **Ask Claude what it did not verify**, and write those items to CLAIMS.md.

Copy-paste task templates:

```
CONTRACT TASK: Implement <contract> per docs/HANDOFF.md Section 6.2. First
write the invariant and unit tests listed for it. Show me failing tests, then
implement. Use OpenZeppelin. No new dependencies without asking.

HSP ADAPTER TASK: Implement <module> in packages/sdk/hsp. Do not guess the
wire format. Read the official HSP docs and the recorded fixtures; if a field
is unknown, stop and ask. Add tests against fixtures.

AUDIT TASK: Act as the solidity-auditor. Review contracts/src against
Sections 6.4 and 9. Output findings with failing tests. Do not edit src.

EVAL TASK: Create the invoice case set in eval/cases.json with the answer key
first. Commit the key. Only then write the runner.

DOCS TASK: Update CLAIMS.md and PROOF.md from the current code and the latest
e2e run. Mark anything not verified on mainnet as simulated or roadmap.
```

12.9 Guardrails (read twice)

- **Secrets.** Never paste private keys or API secrets into prompts. Use a burner deployer key with minimal funds. Keep `.env` out of git and out of Claude's read permissions.
- **Never accept an address from the model.** Token addresses, relayer addresses and chain IDs must be copied from official sources into config, with the source noted.

- **Verify cryptography with test vectors.** EIP-712 hashing, EIP-3009 authorizations and ES256K signing are where silent bugs live. Test against known vectors and against the official HSP examples.
- **Author never approves own work.** Use a separate session or subagent to review.
- **Pin everything.** Solidity version, OpenZeppelin version, Node version, lockfiles.
- **Review every diff** that touches money, signatures or access control.
- **Context hygiene.** Keep CLAUDE.md short, put reference in `docs/`, start a fresh session per milestone, and use the compaction and clear commands described in the docs **[VERIFY names]** when a session gets long.
- **Do not outsource understanding.** If you cannot explain a function to a judge, rewrite or remove it.

---

13. Build plan by layers

Always finish a layer before starting the next. A project that completes L0 and L1 beautifully beats one that half-builds L3.

L0: the lifecycle works on mainnet (the submittable core)

Definition of done:

- Factory, vault, registry and pool deployed **and verified** on HSK mainnet.
- One invoice runs end to end: issue, accept, advance, HSP payment into the vault, waterfall payout, with a transaction hash for each step.
- `verifyInvoice` and `verifyCart` implemented with every refusal code.
- A minimal dashboard shows the timeline and the waterfall.
- The README lets a stranger reproduce the demo on a fork in a few commands.

L1: rigor and judge documents

Definition of done: 70 or more meaningful tests (including fuzz and invariants) passing; fork tests; Slither triaged; eval set run twice with the answer key committed first; AUDIT.md written; JUDGES.md, CLAIMS.md and PROOF.md complete; CI green; demo video recorded.

L2: differentiators

- **Compliance clock:** shows agreed due date against the 45-day legal one per invoice (informational, no legal claim).
- **Evidence pack:** hash-chained event timeline exported as JSON and PDF for disputes.
- **Attack panel:** three live blocked attacks (injected invoice, double financing, payee swap).
- **MCP server** (stdio and HTTP).
- **ReceivableNote (ERC-721):** the lender's claim becomes transferable; the vault pays the current owner.
- **Buyer payment record:** an on-chain view of a buyer's on-time and late history from events.
- **Late and default scenarios** run on mainnet with real transactions.
- **Cross-border framing in the UI:** USD invoice with an INR equivalent at a labelled static rate.

L3: stretch (only when L0–L2 are done)

- Sealed-bid commit-reveal auction between lender agents.
- Senior and junior tranches in the pool.
- ERC-8004 identity and reputation (deploy the reference registries yourselves if absent; accept only settlement-backed feedback).
- EAS attestations for acceptances, if EAS is deployed on HSK **[VERIFY]**.
- A pre-signed, post-dated payment authorization (a digital post-dated cheque). Unknown whether HSP accepts a future `validAfter` **[VERIFY]**; treat as an experiment.
- Chainlink CCIP for cross-chain lender deposits (CCIP is in the organisers' resource list, so it may earn extra points).
- Merkle-tree selective disclosure of invoice fields to lenders.

Suggested schedule (IST; adapt to the real clock)

Phase	What	Target
A. Access and decisions	HSP credentials requested and tested; organisers asked about mainnet; deployer wallet funded with HSK; USDC.e address confirmed from official docs; repo, CLAUDE.md and docs in place; `Terms` and `Acceptance` structs frozen	First 2–3 hours
B. Contracts and tests	Four contracts, invariant tests first	31 Oct evening to night
C. First real HSP payment	A small real payment into a vault address	31 Oct night to 1 Nov morning
D. Agents, desk, keeper, e2e	Seller, buyer, underwriter, webhook, e2e script	1 Nov morning
E. Dashboard, mainnet deploy, PROOF	Verified contracts; real transaction hashes	1 Nov midday
F. Freeze and submit	Feature freeze 15:00; eval, audit fixes, docs, video; submit by 18:30	1 Nov afternoon
G. Polish for Demo Day	L2 and L3 polish. **Check whether submissions can be updated after the deadline; do not assume**	2–4 Nov

Phase C note. In our flow the vault is created right after acceptance (step 4), so at payment time HSP pays an *existing* contract. Still test early that HSP accepts a contract address as `pay_to`. If it does not, fall back to a small receiver wallet that forwards into the vault, and document it.

---

14. Deployment checklist

- **Deployer key:** a fresh burner wallet, funded with just enough HSK for gas. Never a personal wallet.

- **Token address:** copy the USDC.e address from the official HSK token documentation into config and note the source. Never take it from this document or from Claude **[VERIFY]**.
- **Deploy script:** a Foundry script that writes `contracts/deployments/177.json`.
- **Verify on Blockscout** (https://hsk.blockscout.com/) so judges can read the source; check the current `forge verify-contract` flags for that explorer **[VERIFY]**.
- **Desk hosting:** a public HTTPS URL for HSP webhooks, a persistent volume for SQLite, secrets in environment variables, a health endpoint.
- **Web:** host on a static platform; read-only against chain and desk API.
- **Real transactions to record:** at least one on-time, one late and one default lifecycle with small amounts. Paste hashes into PROOF.md.
- **Rail label check:** confirm that settlements funded by HSP's relayer show as *HSP gateway* and anything else as *self-relayed*.
- **Fallbacks for Demo Day:** a pre-recorded video, a seeded dataset, a local fork demo, and a plan for poor venue Wi-Fi.

---

15. The submission package

15.1 DoraHacks BUIDL fields

- **Name:** Hundi.
- **Tagline:** Instant cash against accepted invoices, repaid automatically through HSP.
- **Track:** Agent Payments & A2A Settlement.
- **Problem / solution / how HSP is used / architecture / what is real vs simulated / roadmap.**
- **Links:** public GitHub, demo video, live site, verified contracts, PROOF.md, JUDGES.md.
- **Team:** names and roles.

15.2 README outline

One-paragraph pitch; the problem with numbers and sources; the core insight in one diagram; quick start (fork demo in three commands); architecture; HSP integration, honestly; tests and eval summary; limitations; roadmap; licence.

15.3 JUDGES.md (the five-minute review)

1. What Hundi is, in three lines.
2. How to run the demo in three commands.
3. Where to see the on-chain proof (links).
4. Five things worth checking: the vault address binding, the lender-first waterfall, a blocked injected invoice, a blocked double financing, the test and eval results.
5. Known limits.

15.4 CLAIMS.md

A table: **Claim | Evidence | Status**, where status is *Real on mainnet*, *Real on fork only*, *Simulated* or *Roadmap*. Include the stats and their source quality. Judges reward teams that say what they did not do.

15.5 PROOF.md and AUDIT.md

PROOF.md: contract addresses, a transaction hash for each scenario, the relayer check, test output summary, eval results. **AUDIT.md:** each finding, severity, fix, regression test.

15.6 Demo video (about four minutes)

Time	Content
0:00–0:30	The problem in one stat (₹8.1 lakh crore, 73 vs 45 days) and the one-sentence solution
0:30–1:15	Seller issues a $10,000 invoice; buyer agent verifies; acceptance is signed and recorded
1:15–2:00	Underwriter memo; advance; seller receives $8,820 within seconds; show the transaction
2:00–2:45	Maturity: HSP payment lands in the vault; waterfall pays lender, protocol and seller; amounts match the worked example; rail label shows HSP gateway
2:45–3:30	Three attacks blocked with named refusal codes: injected invoice, double financing, payee swap
3:30–4:00	Proof and rigor: verified contracts, test count, eval results, MCP server; roadmap; closing line

15.7 Demo Day pitch (five minutes)

Slides: (1) the stat; (2) why it persists and the gap TReDS leaves; (3) the insight, the payment path fixed before the loan; (4) live demo; (5) architecture; (6) security and rigor; (7) impact, a per-invoice cost of about $154 on a $10,000 invoice versus 73 days of waiting; (8) roadmap and ask. Keep the stat sources on the slide in small print.

15.8 Hard questions and straight answers

Question	Answer
The statistic is in rupees; why a chain?	The wedge is cross-border export invoices, which settle in stablecoins. The rupee figure sizes the pain; it is not the settlement currency

Question	Answer
Is this lending? Does it need a licence?	This is a prototype on a public chain with no legal claims. A production version would operate through licensed partners; we say so in CLAIMS.md
How is this different from TReDS?	It complements TReDS for buyers and invoices outside that platform, including cross-border, and the payment path is cryptographically fixed
What if the buyer refuses to pay?	The lender's loss is limited by the advance rate and the seller's bond; default is declared on-chain, the evidence pack is generated and the buyer's record is marked
How do you know the goods were delivered?	Prototype uses a signed delivery attestation. Honest limit. Roadmap is GST e-invoice and e-way bill data
What stops the AI being tricked?	The AI never approves. Deterministic code does, and we show three live attacks blocked plus an eval set with a committed answer key
What does HSP add over a plain USDC transfer?	It is the official settlement path: merchant-signed carts, wallet-signed mandates, a relayer and webhooks. We bind its `pay_to` to the loan terms
Who are the lenders and why lend?	Pool depositors earn the discount fee for short, buyer-backed exposure with caps. Return and risk are parameters, not promises
Who holds keys? Are contracts upgradeable?	No proxies. Settlement logic is permissionless. The only privileged role is a guardian that can pause new advances
Why not an escrow like Proviso?	Escrow protects a buyer from non-delivery. We create credit for the seller. The two are complementary
What is simulated?	See CLAIMS.md: delivery attestation, FX rate, and any HSP step run in self-relay mode

Question	Answer
Is ₹8.1 lakh crore reliable?	It is the Economic Survey estimate as reported by several outlets; we cite the source and treat vendor figures as such

---

16. Risks, honest limits and open questions

Risk or question	Plan
HSP credentials or sandbox not available in time	Build to the wire spec behind one interface; label self-relayed; keep asking organisers
Mainnet rule conflicts with the event text	Ask organisers; plan for mainnet anyway
HSP cart expiry unknown	Create carts just-in-time at maturity
`pay_to` as a contract may be restricted	Test on day one; fallback forwarder
USDC.e address and relayer address unverified	Copy from official docs; verify relayer on-chain
Stablecoin vs rupee mismatch	Lead with cross-border export invoices
Lending licensing and regulation	No legal claims; say prototype; roadmap with licensed partners
Delivery attestation trust	State as limitation
One merchant of record	State as limitation; roadmap per-seller merchants
Pool solvency and lender risk	Caps, bounds, bond, honest disclosure
Strong competitors (especially Proviso)	Win on category and problem scale; match their rigor
Scope creep	Layers; never start L2 before L0 and L1 are done
LLM errors and invented facts	Deterministic gates; verify every protocol claim and address
Statistics from secondary sources	Verify primary sources; label vendor data

---

17. Sources

Hackathon and HSK Chain: official Mumbai page https://dorahacks.io/hackathon/hskchainindia/detail · HSP API docs https://hashkey-merchant.github.io/en/index.html · HashFans https://hashfans.io/ · HSK docs

https://hskchain.net/docs · explorer https://hsk.blockscout.com/ · faucet https://hskchain.net/faucet.

Past winners: Hong Kong report https://dorahacks.io/hackathon/2045/report · Japan edition https://dorahacks.io/hackathon/hskchainjapan · Japan winners post https://x.com/HSKChain/status/2077272198910529876.

Competitor repos (HSP facts in this document came largely from the first): https://github.com/RaYYeR220/proviso · https://github.com/devx-agent/devx-agenttab · https://github.com/Nouman-wp/Kohaku · https://github.com/sagarjethi/payminto-clearing · https://github.com/mesh-pch/leash-hashkey · https://github.com/kanakagarwal0901/HashKey-Chain-Horizon · https://github.com/rohitshukla11/likeness-hsk.

Problem statistics (verify primary sources): https://sriramananthakrishnan.substack.com/p/indias-digital-finance-revolution · https://fintechbiznews.com/fintech-information-technology/risk-to-receivables-how-indian-smes-are-struggling · https://msmesampark.com/ · https://finbox.substack.com/p/can-treds-pull-some-threads-to-capture.

Agent security: https://www.ccn.com/news/crypto/hackers-malicious-websites-ai-agents-crypto-payments/ · https://chainpursuit.com/blog/ai-agent-crypto-scams/.

Standards: ERC-8004 https://eips.ethereum.org/EIPS/eip-8004 · reference contracts https://github.com/erc-8004/erc-8004-contracts.

Claude Code: docs index https://code.claude.com/docs/llms.txt · workflows https://code.claude.com/docs/en/common-workflows.md · community guides (verify against official docs): https://github.com/shanraisshan/claude-code-best-practice and https://community.sap.com/t5/artificial-intelligence-blogs-posts/claude-code-best-practices-for-developers/ba-p/14394164.

End of handoff. When something here conflicts with an official document, the official document wins; record the correction in `docs/DECISIONS.md`*.*